

Drone Technology and Robotics

EE-304

By-
Dr. Pinku Ranjan
Assistant Professor
ABV-IIITM GWALIOR, INDIA

Slide-6

For B.Tech (Open Elective)

Planning and Decision Making

- Motion planning problem formulation
- Graph-based planning
- Sampling-based planning
- Trajectory generation
- Decision-making under uncertainty
- Introduction to Markov Decision Processes (MDP)

Planning and Decision Making

- Planning and decision making are essential components of **autonomous systems** such as robots, drones, and self-driving vehicles. These systems must determine **how to move from a current state to a desired goal while avoiding obstacles and optimizing performance.**
- **Planning** refers to generating a sequence of actions or motions that lead from the **initial state to the goal state.**
- **Decision making** involves selecting the **best action among several alternatives**, often in uncertain or dynamic environments.
- Safety (avoid collisions)
- Efficiency (minimum time, energy, or distance)
- Adaptability to environmental changes

In robotics, planning and decision making usually rely on **mathematical models, algorithms, and sensor data.**

Motion Planning Problem Formulation

Motion planning determines a **collision-free path** for a robot from a starting position to a target position.

Components of Motion Planning

1.Configuration Space (C-space)

1. Represents all possible states or positions of the robot.
2. Each point corresponds to a valid configuration of the robot.

2.Obstacle Space

1. Regions in configuration space where the robot cannot move due to obstacles.

3.Free Space

1. The portion of configuration space where the robot can move safely.

4.Start Configuration

1. The initial state of the robot.

5.Goal Configuration

1. The desired final state.

Motion Planning Objective

✓ Find a **continuous path from start to goal configuration that avoids obstacles.**

Typical constraints include:

- Kinematic constraints
- Dynamic constraints
- Energy consumption
- Environmental uncertainties

Motion planning is widely used in:

- Autonomous vehicles
- Drone navigation
- Industrial robotics
- Warehouse automation

Graph-Based Planning

- ✓ Graph-based planning represents the environment as a **graph structure**.

Graph Representation

- **Nodes (Vertices)** represent possible states or positions.
- **Edges** represent possible movements between states.

The planning problem becomes a **graph search problem**.

Common Graph-Based Algorithms

1. Dijkstra's Algorithm

- Finds the **shortest path between nodes in a weighted graph**.
- Guarantees the optimal path.

A* Algorithm

- Uses **heuristic information** to speed up path search.
- One of the most widely used algorithms in robotics and games.

Formula used in A*:

$$f(n) = g(n) + h(n)$$

Where:

- **$g(n)$** = cost from start to node n
- **$h(n)$** = estimated cost from n to goal

Advantages

- Guarantees optimal solutions.
- Efficient for structured environments like grids.

Limitations

- High computational cost in large spaces.
- Not suitable for high-dimensional robots.

Sampling-Based Planning

- ✓ Sampling-based planning is used when the **configuration space is very large or complex**.
- ✓ Instead of searching the entire space, the algorithm **randomly samples possible configurations**.
- ✓ Build a path using randomly sampled points in free space.

Common Algorithms

1. Probabilistic Roadmap (PRM)

Steps:

1. Randomly sample points in free space.
2. Connect nearby points with edges.
3. Build a roadmap graph.
4. Search for a path using graph search.

Applications:

- Multi-query motion planning
- Static environments

2. Rapidly-exploring Random Tree (RRT)

RRT builds a **tree that expands toward unexplored regions**.

Steps:

1. Start from the initial configuration.
2. Randomly sample a point.
3. Extend the tree toward the sampled point.
4. Repeat until the goal is reached.

Advantages:

- Efficient in high-dimensional spaces
- Suitable for complex robots

Limitations:

- Paths are often **not optimal**
- Requires smoothing

Trajectory Generation

A **trajectory** is a time-based path describing how a robot moves from start to goal.

Difference Between Path and Trajectory

Path	Trajectory
Spatial description	Spatial + time description
Only position	Position, velocity, acceleration

Trajectory generation ensures:

- Smooth motion
- Feasible robot dynamics
- Safe movement

Typical Methods

1. Polynomial Trajectories

1. Cubic or quintic polynomials
2. Smooth velocity and acceleration

2. Spline Interpolation

1. Connects path segments smoothly

3. Optimization-Based Trajectories

1. Minimizes cost (energy, jerk, time)

Example: Drone flight planning uses trajectories to ensure: smooth turns, stable flight, minimal energy consumption

Decision Making Under Uncertainty

In real environments, robots face **uncertainty** due to:

- Sensor noise
- Dynamic obstacles
- Incomplete environmental knowledge

Decision-making systems must choose actions **even when outcomes are uncertain**.

1.Probabilistic Models

2.Bayesian inference

3.Markov models

4.Reinforcement learning

Example: An autonomous vehicle may not know exactly where pedestrians are due to sensor noise. The system must still decide whether to **slow down, stop, or continue moving**.

Introduction to Markov Decision Processes (MDP)

- ✓ A **Markov Decision Process (MDP)** is a mathematical framework used for **decision making in stochastic environments**.
- ✓ It models situations where **actions lead to probabilistic outcomes**.

Components of an MDP

An MDP is defined by a tuple:

$$(S, A, P, R, \gamma)$$

Where:

Symbol	Meaning
S	Set of states
A	Set of actions
P	State transition probability
R	Reward function
γ	Discount factor

Markov Property

The next state depends only on the current state and action, not on previous history.

Working Principle

1. The system is in a state.
2. An action is chosen.
3. The environment moves to a new state with some probability.
4. A reward is received.

Goal:

Maximize the expected cumulative reward.

Example: Drone Navigation

State: drone position

Action: move forward, turn left, turn right

Reward: reaching goal (+10), hitting obstacle (-10)

The drone learns a **policy** that maximizes long-term reward.

Solution Methods

Common algorithms include:

- Value Iteration
 - Policy Iteration
 - Q-Learning
 - Deep Reinforcement Learning
-

End of Slide-6